

28 Fase 3 - Analisador Semântico

Este trabalho pode ser realizado em grupos de até 4 alunos. Grupos com mais de 4 membros terão o trabalho anulado. Leia todo este documento antes de começar e siga o seguinte código de ética: você pode discutir as questões com colegas, professores e amigos, consultar livros da disciplina, bibliotecas virtuais ou físicas, e a Internet em geral, em qualquer idioma. Com ou sem o uso de ferramentas de Inteligência Artificial. Contudo, o trabalho é seu e deve ser realizado por você. Plágio, ou qualquer outra violação das regras de integridade acadêmica, resultará na anulação do trabalho.

Os trabalhos entregues serão avaliados por uma ferramenta de Inteligência Artificial, que verificará a originalidade do texto e a autoria do código e o cumprimento de todas as regras de entrega e desenvolvimento. Para isso, o trabalho deve ser entregue em um repositório público no GitHub. O resultado desta avaliação automatizada será a nota base do trabalho, que poderá ser alterada durante a prova de autoria (ver o plano de ensino).

Este é um projeto “Melhorado com o uso de Ferramentas de Inteligência Artificial”. O uso de ferramentas de Inteligência Artificial não é apenas permitido, mas incentivado. No entanto, seu uso é governado por um princípio fundamental: **Você é o arquiteto e o engenheiro; a Inteligência Artificial é sua assistente. Você mantém total responsabilidade pelo código que envia.** Você poderá usar as ferramentas de Inteligência Artificial para:

- Gerar código repetitivo (*boilerplate*) e arquivos de configuração;
- Explicar conceitos complexos e documentação;
- Depurar mensagens de erro e sugerir correções;
- Escrever testes unitários e documentação;
- Otimizar e refatorar código existente;
- Fazer *brainstorming* de ideias de projetos e arquitetura.

Não é permitido:

- **Envio às Cegas:** enviar código gerado por uma Inteligência Artificial sem revisão, compreensão ou modificação é uma violação da integridade acadêmica. Você deve ser capaz de explicar qualquer código do seu projeto, linha por linha se for necessário. Este tipo de envio provocará a anulação do trabalho.
- **Contornar o Aprendizado:** usar Inteligência Artificial para completar os objetivos centrais de aprendizado por você é proibido, por exemplo, pedir a uma Inteligência Artificial para construir um componente inteiro do projeto que você tem a tarefa de aprender. Esta prática é considerada uma forma de trapaga e resultará na anulação do trabalho.
- **Violação de Licenças:** é proibido enviar código que viole as políticas de honestidade acadêmica da universidade, a licença dos dados de treinamento, ou qualquer outra restrição legal. Qualquer indicativo desta prática resultará na anulação do trabalho.

28.1 1. Objetivo

Pesquisar e praticar os conceitos de analisadores léxico, sintático e semântico usando apenas as linguagens **Python**, **C**, ou **C++**. Este trabalho irá complementar o material visto em aula, aprimorando sua formação acadêmica e profissional. Desta vez você deverá incluir

a análise semântica ao processo de compilação que estamos desenvolvendo. Sua tarefa, em resumo, será criar a gramática aumentada e, conseqüentemente, a árvore sintática aumentada usando os resultados que obtemos antes na Fase 1 e 2. A partir da árvore sintática, você deverá gerar o código Assembly para o ambiente **Cpulator-ARMv7 DEC1-SOC(v16.1)**. O código Assembly deve ser gerado diretamente a partir da árvore sintática aumentada, sem a necessidade de uma representação intermediária.

28.2 2. Descrição do Trabalho

O objetivo é desenvolver um programa capaz de:

1. Ler um arquivo de texto contendo o código-fonte de um programa escrito na linguagem especificada neste documento (a linguagem que você expandiu na **Fase 2**);
2. Utilizar o *string* de *tokens* (gerado pelo analisador léxico, como o da Fase 1) como entrada para o analisador sintático;
3. Utilizar a árvore sintática gerada na **Fase 2** como base para a construção da árvore sintática aumentada, incorporando as regras semânticas necessárias para a geração de código Assembly;
4. Detectar e reportar erros léxicos, sintáticos e semântico de forma clara e informativa.

28.2.1 2.1. Características da Linguagem

Nesta fase, vamos incluir comentários na nossa linguagem. Usaremos `*{` para iniciar um comentário e `}` para finalizar um comentário. Os comentários podem estar em qualquer lugar do código. Além disso, as variáveis da nossa linguagem devem ser definidas, antes de serem usadas em expressões ou comandos. A definição de uma variável é feita usando o comando (**V MEM**), onde **MEM** é o nome da memória (variável) que está sendo definida. Após a definição, a variável pode ser usada em expressões e comandos subsequentes, e seu valor pode ser recuperado usando o comando (**MEM**).

Note

Nota: você não precisa criar uma gramática para lidar com os comentários, mas deve garantir que o analisador léxico da Fase 1 seja capaz de identificar e ignorar os comentários corretamente, sem que eles interfiram na análise sintática ou semântica. O analisador léxico deve tratar os comentários como tokens de tipo “comentário” e descartá-los durante a geração do vetor de tokens que será usado pelo analisador sintático.

Como cada grupo implementou uma linguagem diferente na **Fase 2**, a linguagem a ser usada nesta fase é aquela que cada grupo definiu e implementou. No entanto, as características gerais da linguagem, que foram definidas nas Fases 1 e 2, devem ser mantidas. Estas características incluem:

Precisão Numérica:

Todos os valores reais serão tratados como números de ponto flutuante de precisão dupla (double) segundo a norma IEEE 754. As operações de divisão inteira e resto são realizadas exclusivamente com números inteiros.

Expressões Aninhadas:

Expressões podem ser aninhadas sem limite definido, por exemplo:

- $(A \ (C \ D \ *) \ +)$: Soma A ao produto de C e D;
- $((A \ B \ \%) \ (D \ E \ *) \ /)$: Divide o resto de A por B pelo produto de D e E;

- $((A \ B \ +) \ (C \ D \ *) \ |)$: Divide (usando divisão real) a soma de **A** e **B** pelo produto de **C** e **D**.

Nestes exemplos, **A**, **B**, **C**, **D**, e **E** podem ser números literais ou outras operações RPN.

Caracterização como Programa:

Atenção: deste ponto em diante, estamos escrevendo um programa completo e não um conjunto de expressões. Todos os programas devem começar com a linha (**START**), e terminar com a linha (**END**). O programa pode conter expressões RPN, comandos especiais, estruturas de controle, ou qualquer combinação destes, desde que estejam entre as linhas (**START**) e (**END**).

28.2.2 2.2. Comandos Especiais

A linguagem inclui os mesmos comandos especiais da fase anterior para manipulação de memória e resultados:

- (**N RES**): Retorna o resultado da expressão **N** linhas anteriores (**N** é um inteiro não negativo);
- (**V MEM**): Armazena o valor real **V** em uma memória chamada **MEM**;
- (**MEM**): Retorna o valor armazenado em **MEM**. Se a memória não foi inicializada, retorna 0.

Regras de Escopo:

- Cada arquivo de texto representa um escopo de memória independente;
- **MEM** pode ser qualquer conjunto de letras latinas maiúsculas (ex: **MEM**, **VAR**, **X**, **CONTADOR**);
- **RES** é uma keyword da linguagem.

28.2.3 2.3. Validação de Tipos

Você deverá criar e documentar as regras de validação de tipos para a linguagem, garantindo que as operações sejam realizadas entre tipos compatíveis. A partir deste ponto, a linguagem deve ser capaz de lidar com tipos de dados, e o analisador semântico deve incorporar regras semânticas para validar as operações entre esses tipos. Os tipos que usaremos serão: inteiros, reais, e lógicos (**bool**). As regras de validação de tipos devem ser claramente definidas e documentadas em cálculo de sequentes.

A definição de tipos em artefatos não literais será inferida a partir do contexto, ou seja, a partir do tipo dos operandos usados em expressões e comandos. Por exemplo, se **A** é usado em uma expressão com um número real, **A** será inferido como um número real. Se **B** é usado em uma expressão com um número inteiro, **B** será inferido como um número inteiro. Se **C** é usado em uma expressão com um valor lógico, **C** será inferido como um valor lógico. O analisador semântico deve ser capaz de realizar essa inferência de tipos e validar as operações de acordo com as regras de compatibilidade de tipos definidas.

Finalmente, os tipos que usaremos na nossa linguagem são estáticos e fortes, ou seja, o tipo de cada variável, ou artefato, é determinado no momento da definição e não pode ser alterado posteriormente, e as operações entre tipos incompatíveis devem resultar em erros semânticos.

28.2.4 2.4. Geração de Assembly

Nesta fase, a geração de código Assembly para o ambiente **Cpulator-ARMv7 DEC1-SOC(v16.1)** será feita diretamente a partir da árvore sintática aumentada gerada pelo analisador semântico, sem a necessidade de uma representação intermediária.

A geração de código Assembly deve ser feita após a construção da árvore sintática aumentada, ou seja, à medida que o *parser* processa as expressões e comandos, ele deve gerar o código Assembly correspondente.

28.3 3. Analisador Semântico a partir do *parser* $LL(1)$

Aproveite o *parser* usado na **Fase 2** para a criação da árvore sintática aumentada. A implementação deve incluir:

28.3.1 3.1. Gramática $LL(1)$

Definir o conjunto completo de regras de produção para a linguagem, incluindo, mas não se limitando a:

- Expressões aritméticas no formato RPN que definimos nas fases anteriores;
- Comandos especiais (**RES**, **MEM**);
- Estruturas de controle (decisão e laços);
- Tratamento de aninhamento.

Note

Nota: A gramática deve ser escrita em formato EBNF, **usando letras minúsculas para não-terminais e maiúsculas para terminais**. Esta será uma gramática aumentada, que inclui as regras semânticas necessárias para a geração de código Assembly. A gramática deve ser clara, completa e livre de ambiguidades, garantindo que seja adequada para um *parser* $LL(1)$.

28.3.2 3.5. Funções de Teste

Criar funções de teste específicas para validar o analisador semântico, cobrindo:

- Erros léxicos (tokens inválidos);
- Erros sintáticos (expressões mal formadas);
- Erros semânticos (tipos incompatíveis, uso incorreto de variáveis);
- Estruturas de controle válidas;
- Erros semânticos em estruturas de controle (condições inválidas, uso incorreto de variáveis em laços);
- Erros de tipos em expressões aninhadas;
- Casos extremos (aninhamento profundo, expressões vazias);

Important

Atenção: O analisador sintático deve ser robusto e capaz de lidar com uma variedade de entradas, incluindo casos de erro. A qualidade da implementação do *parser* e a clareza das mensagens de erro serão critérios importantes na avaliação. Além disso: **durante a prova de autoria os testes serão feitos com um código-fonte diferente dos arquivos de teste entregues, e fornecido pelo professor, para validar a generalização do analisador sintático.**

28.4 4. Arquivos de Teste

Fornecer **um mínimo de 3 arquivos de texto**, cada um com pelo menos **10 linhas** de expressões segundo as especificações deste documento.

Requisitos dos Arquivos de Teste:

- Cada arquivo deve incluir todas as operações (+, -, *, |, /, %, ^);
- Cada arquivo deve incluir todos os comandos especiais ((N RES), (V MEM), (MEM));
- Cada arquivo deve incluir pelo menos um laço de repetição;
- Cada arquivo deve incluir pelo menos uma tomada de decisão;
- Cada arquivo deve incluir todos os tipos e todas as regras de combinação possíveis entre estes tipos para operações e artefatos de decisão e repetição;
- Cada arquivo deve incluir expressões aninhadas de complexidade crescente, sem limite definido;
- Cada arquivo deve incluir comentários usando a sintaxe definida (*{ comentário }*). Em comentários de linhas inteiras, comentários no final de linhas de código, e comentários entre expressões;
- Os testes devem incluir literais inteiros, reais, lógicos e o uso de memórias (variáveis);
- Incluir casos de teste com erros léxicos, sintáticos e semânticos para validar o tratamento de erros.

Os arquivos devem estar no mesmo diretório do código-fonte e ser processados via argumento de linha de comando (ex.: `./AnalizadorSemantico teste1.txt`). Este comando deve rodar os três analisadores e fornecer as saídas correspondentes, incluindo a árvore sintática gerada e o código Assembly.

28.5 5. Hospedagem no GitHub

O projeto deve ser hospedado em um repositório público no GitHub, nomeado segundo o nome do grupo no ambiente virtual de aprendizado, **criado exclusivamente para este projeto**. O repositório deve ser criado por um dos alunos do grupo. Este aluno, e apenas este aluno, será o administrador do repositório, e os outros membros do grupo não terão acesso de escrita ao repositório. O administrador do repositório é responsável por integrar as contribuições dos outros membros do grupo, garantindo que o código final seja coeso e funcional.

O repositório deve conter:

- Código-fonte do programa (**Python**, **C**, ou **C++**);
- Arquivos de teste (mínimo 3);
- Funções de teste para o analisador sintático;
- Documentação completa (README.md);
- Arquivo markdown com a árvore sintática atribuída;
- Arquivo markdown contendo a gramática atribuída em formato EBNF;
- Arquivo markdown contendo o sistema de regras de validação de tipos em cálculo de seguintes;
- Arquivo markdown contendo a tabela de símbolos;
- Os últimos resultados da execução do programa, incluindo a árvore sintática gerada.

O repositório deve ser organizado com *commits* claros, as contribuições de cada um dos alunos devem estar registradas na forma de *pull requests*.

! Important

- O nome do repositório deve ser o nome do grupo no Canvas (ex.: RA2_1);
- É fundamental que cada aluno documente suas contribuições de forma clara e detalhada;
- O aluno não pode trocar o seu usuário no GitHub durante o desenvolvimento do projeto;
- O aluno não pode alterar o repositório para privado;
- O uso de *issues* no GitHub é encorajado para discutir tarefas e bugs;
- Um e somente um aluno do grupo será o administrador do repositório. Nenhum outro aluno do grupo pode ter acesso de escrita ao repositório, para evitar confusão e garantir a rastreabilidade das contribuições. O administrador do repositório é responsável por integrar as contribuições dos outros membros do grupo, garantindo que o código final seja coeso e funcional.
- O desbalanceamento da contribuição entre os membros do grupo, ou a falta de documentação das contribuições, pode resultar na perda de 20% da nota parcial do trabalho (nota antes da prova de autoria).
- Alterações no repositório que violem as regras de integridade acadêmica, que sejam consideradas como plágio ou que tenham sido realizadas depois da data limite de entrega resultarão na anulação do trabalho (nota final zero).

28.6 6. Requisitos do Código

As primeiras linhas do código devem conter:

```
// Integrantes do grupo (ordem alfabética):  
// Nome Completo 1 - username1  
// Nome Completo 2 - username2  
// Nome Completo 3 - username3  
// Nome Completo 4 - username4  
//  
// Nome do grupo no Canvas: [Nome do Grupo]
```

O programa deve receber o nome do arquivo de teste como argumento na linha de comando. Nenhum menu será implementado. O programa deve ser executado com, por exemplo: `./AnalizadorSemantico teste1.txt`.

O código deve ser escrito em **Python**, **C**, ou **C++**, com as funções nomeadas como está explicitado na Seção 7.

28.7 7. Divisão de Tarefas para a Fase 3

O trabalho será dividido entre até quatro alunos, trabalhando independentemente, na mesma sala, ou de forma remota. Cada aluno será responsável por uma parte específica do analisador semântico, com interfaces claras para facilitar a integração. Abaixo está uma **sugestão de divisão das tarefas**, considerando as funções solicitadas: `prepararEntradaSemantica`, `construirTabelaSimbolos`, `verificarTipos`, `gerarArvoreAtribuida` e `gerarAssembly`.

i Note

Nota: As tarefas podem ser divididas da forma que cada grupo achar mais conveniente, desde que as funções e interfaces sejam respeitadas e que o trabalho esteja equitativamente distribuído entre os membros do grupo.

Nota: A árvore sintática atribuída, a tabela de símbolos e o relatório de erros semânticos devem ser salvos em formato texto, JSON ou Markdown. Estes artefatos farão parte da avaliação e devem estar documentados no repositório.

Nota: A **Fase 3** deve reaproveitar o analisador léxico e o analisador sintático das fases anteriores, mas agora acrescentando informações semânticas: tipos, declarações, usos de variáveis, compatibilidade entre operandos, validade de condições e dados necessários para a geração correta de Assembly.

28.7.1 7.1. Aluno 1: Função `prepararEntradaSemantica` e Integração com as Fases Anteriores

Responsabilidades:

- Implementar `prepararEntradaSemantica(arquivo)` para carregar o programa-fonte, remover ou descartar comentários e preparar os tokens que serão usados pelo analisador sintático e semântico;
- Adaptar o analisador léxico da Fase 1 para reconhecer comentários iniciados por `*{` e terminados por `};`;
- Garantir que comentários possam aparecer em linhas inteiras, no final de linhas de código e entre expressões;
- Reaproveitar o analisador sintático da Fase 2 para produzir a árvore sintática que será aumentada nesta fase;
- Validar se o programa começa com (START) e termina com (END);
- Criar testes para comentários, tokens válidos, tokens inválidos e integração com os arquivos gerados nas fases anteriores.

Tarefas Específicas:

- Ler o arquivo de entrada recebido por argumento de linha de comando;
- Produzir o vetor de tokens sem comentários;
- Acionar o analisador sintático da Fase 2;
- Gerar uma árvore sintática inicial válida para uso pelo analisador semântico;
- Reportar erros léxicos e sintáticos antes da etapa semântica.

Interface:

- Entrada: nome do arquivo de teste;
- Saída: vetor de tokens e árvore sintática inicial;
- Fornece a árvore sintática para `construirTabelaSimbolos()` e `gerarArvoreAtribuida()`.

28.7.2 7.2. Aluno 2: Função `construirTabelaSimbolos` e Validação de Declarações

Responsabilidades:

- Implementar `construirTabelaSimbolos(arvore)` para percorrer a árvore sintática e registrar variáveis, tipos inferidos, linhas de declaração e linhas de uso;
- Verificar se toda variável foi definida antes de ser usada;

- Controlar o significado semântico dos comandos especiais (V MEM), (MEM) e (N RES);
- Registrar o tipo de cada variável no momento de sua definição;
- Impedir que uma variável já definida com um tipo seja usada posteriormente como se tivesse outro tipo;
- Gerar mensagens de erro semântico claras, indicando linha, variável e causa do erro.

Tarefas Específicas:

- Criar uma estrutura de tabela de símbolos;
- Registrar identificadores, tipos, escopo, linha de definição e linha de último uso;
- Verificar uso de variável não declarada;
- Verificar redefinições incompatíveis;
- Validar referências a resultados anteriores feitas com RES;
- Salvar a tabela de símbolos em arquivo Markdown, texto ou JSON.

Interface:

- Entrada: árvore sintática inicial;
- Saída: tabela de símbolos e lista de erros semânticos relacionados a declarações;
- Fornece a tabela de símbolos para `verificarTipos()` e `gerarArvoreAtribuida()`.

28.7.3 7.3. Aluno 3: Função `verificarTipos` e Sistema de Regras Semânticas

Responsabilidades:

- Implementar `verificarTipos(arvore, tabelaSimbolos)` para validar os tipos das expressões, comandos especiais, decisões e laços;
- Definir e documentar as regras de tipos da linguagem usando cálculo de seqüentes;
- Verificar operações entre inteiros, reais e valores lógicos (`bool`);
- Garantir que divisão inteira e resto sejam usados apenas com inteiros;
- Garantir que divisão real, soma, subtração, multiplicação e potenciação sejam usadas apenas com operandos compatíveis;
- Garantir que condições de decisão e repetição tenham tipo lógico;
- Criar testes para expressões válidas e inválidas envolvendo todos os tipos.

Tarefas Específicas:

- Definir regras formais para literais inteiros, reais e lógicos;
- Definir regras formais para operadores aritméticos, relacionais e lógicos;
- Implementar inferência de tipos para variáveis a partir de suas definições e usos;
- Detectar incompatibilidades de tipos em expressões aninhadas;
- Produzir um relatório de erros semânticos;
- Salvar em Markdown o sistema de regras de tipos usado pelo grupo.

Interface:

- Entrada: árvore sintática inicial e tabela de símbolos;
- Saída: tipos inferidos para os nós da árvore e lista de erros semânticos;
- Fornece informações de tipo para `gerarArvoreAtribuida()`.

28.7.4 7.4. Aluno 4: Funções gerarArvoreAtribuida, gerarAssembly, Interface e Testes Finais

Responsabilidades:

- Implementar `gerarArvoreAtribuida(arvore, tabelaSimbolos, tipos)` para produzir a árvore sintática aumentada;
- Implementar `gerarAssembly(arvoreAtribuida)` para gerar código Assembly compatível com o CpuLator-ARMv7 DEC1-SOC(v16.1);
- Implementar ou adaptar a função `main()` para coordenar a execução completa do compilador;
- Integrar as funções dos demais alunos;
- Criar testes de ponta a ponta usando os três arquivos de teste exigidos;
- Garantir que os artefatos finais sejam salvos no repositório.

Tarefas Específicas:

- Anotar cada nó relevante da árvore com tipo, categoria semântica e informação necessária para geração de código;
- Gerar Assembly apenas para programas sem erros léxicos, sintáticos ou semânticos;
- Produzir mensagens claras quando a geração de Assembly for interrompida por erro semântico;
- Salvar a árvore sintática atribuída em arquivo Markdown, texto ou JSON;
- Salvar o código Assembly correspondente à última execução;
- Testar o programa completo com casos válidos e inválidos.

Interface:

- Entrada: árvore sintática inicial, tabela de símbolos e tipos inferidos;
- Saída: árvore sintática atribuída, código Assembly e relatório final de execução;
- Gerencia a execução completa via linha de comando.

28.8 8. Considerações para Integração

Interfaces: O grupo deve definir, antes da implementação, o formato dos tokens, da árvore sintática, da tabela de símbolos, da lista de erros e da árvore sintática atribuída.

Depuração: Cada módulo deve ser testado isoladamente antes da integração. O grupo deve criar testes específicos para comentários, declarações, uso de variáveis, inferência de tipos, incompatibilidades semânticas, estruturas de controle e geração de Assembly.

Passos de Integração:

1. Integrar `prepararEntradaSemantica()` com o analisador léxico e sintático das fases anteriores;
2. Integrar `construirTabelaSimbolos()` à árvore sintática gerada;
3. Integrar `verificarTipos()` à tabela de símbolos;
4. Integrar `gerarArvoreAtribuida()` com as informações semânticas;
5. Integrar `gerarAssembly()` apenas após a validação semântica completa;
6. Executar os três arquivos de teste e salvar os artefatos finais.

Resolução de Conflitos: Problemas de integração devem ser registrados em *issues* ou *pull requests* no GitHub, com indicação clara do erro, da função afetada e da solução adotada.

Depuração Final: O programa completo deve ser testado com arquivos válidos e inválidos, verificando erros léxicos, sintáticos e semânticos, além da geração correta da árvore sintática atribuída e do código Assembly.

28.9 9. Avaliação

O trabalho será pré-avaliado de forma automática e novamente durante a prova de autoria, com os seguintes critérios. A avaliação desta fase considera que o grupo já possui um analisador léxico e um analisador sintático funcionais, desenvolvidos nas fases anteriores. O foco agora é a **análise semântica**, a construção da **árvore sintática atribuída**, a validação de tipos, o uso correto da tabela de símbolos e a geração de Assembly a partir de um programa semanticamente válido.

28.9.1 9.1. Funcionalidades do Analisador Semântico (70%)

- Integração correta com o analisador léxico e sintático das fases anteriores;
- Tratamento correto de comentários iniciados por `*{` e terminados por `}*;`
- Construção correta da tabela de símbolos;
- Detecção de uso de variável antes da definição;
- Registro correto de identificadores, tipos, linhas de definição e linhas de uso;
- Inferência correta de tipos para inteiros, reais e lógicos (`bool`);
- Validação correta de operações aritméticas, relacionais e lógicas;
- Rejeição de operações entre tipos incompatíveis;
- Validação semântica de comandos especiais como (`V MEM`), (`MEM`) e (`N RES`);
- Validação semântica de estruturas de decisão e repetição;
- Geração correta da árvore sintática atribuída;
- Geração de Assembly apenas para programas semanticamente válidos;
- Mensagens de erro claras, indicando linha, elemento envolvido e causa do erro.

As penalidades abaixo serão aplicadas sobre os 70% desta seção:

- Comentários não reconhecidos ou tratados incorretamente: **-15%**;
- Ausência de tabela de símbolos: **-40%**;
- Tabela de símbolos incompleta ou sem tipos registrados: **-25%**;
- Não detectar uso de variável antes da definição: **-30%**;
- Não detectar redefinição ou uso incompatível de tipos: **-25%**;
- Ausência de regras formais de tipos em cálculo de sequentes: **-30%**;
- Regras formais documentadas, mas incompatíveis com a implementação: **-20%**;
- Falha na inferência de tipos: **-30%**;
- Aceitar operações entre tipos incompatíveis: **-30%**;
- Rejeitar programas semanticamente válidos: **-20%**;
- Não validar condições de decisão e repetição como expressões lógicas: **-25%**;
- Não gerar árvore sintática atribuída: **-40%**;
- Gerar árvore sintática atribuída sem anotações de tipo: **-25%**;
- Gerar Assembly mesmo quando há erros semânticos: **-40%**;
- Não gerar Assembly para programas semanticamente válidos: **-30%**;
- Assembly gerado, mas incompatível com a árvore sintática atribuída: **-30%**;
- Mensagens de erro ausentes, genéricas ou sem indicação de linha: **-20%**.

! Important

Atenção: Se o programa aceitar como válido um código-fonte com erro semântico evidente, como uso de variável não declarada, operação aritmética com valor lógico, condição de laço não lógica, ou uso incompatível de uma variável previamente tipada, a nota desta seção poderá ser severamente reduzida, mesmo que o analisador sintático funcione corretamente.

28.9.2 9.2. Documentação, Organização e Legibilidade do Código (15%)

- Código claro, comentado e bem estruturado;
- Separação adequada entre análise léxica, análise sintática, análise semântica e geração de Assembly;
- README bem escrito contendo:
 - Nome da instituição de ensino, ano, disciplina e professor;
 - Integrantes do grupo em ordem alfabética;
 - Instruções para compilar, executar e depurar;
 - Descrição da linguagem implementada pelo grupo;
 - Descrição dos tipos suportados;
 - Exemplos de programas válidos e inválidos;
 - Explicação da tabela de símbolos;
 - Explicação da árvore sintática atribuída;
- Documentação da gramática atribuída em EBNF;
- Documentação das regras semânticas em cálculo de sequentes;
- Repositório GitHub organizado com *commits* e *pull requests* claros.

As penalidades abaixo serão aplicadas sobre os 15% desta seção:

- README ausente ou insuficiente: **-50%**;
- Ausência da gramática atribuída documentada: **-40%**;
- Ausência das regras semânticas em cálculo de sequentes: **-50%**;
- Código sem separação clara entre módulos: **-30%**;
- Repositório desorganizado ou sem artefatos finais: **-30%**;
- Contribuições dos alunos sem rastreabilidade por *pull requests*: **-30%**.

i Note

Nota: qualquer falha grave na organização e legibilidade do código pode implicar perda total dos 15% desta seção, mesmo que o programa funcione corretamente. A clareza da implementação é parte da avaliação, pois o grupo deverá defender o funcionamento do analisador durante a prova de autoria.

28.9.3 9.3. Robustez, Testes e Tratamento de Erros (15%)

- Testes cobrindo programas válidos e inválidos;
- Testes com erros léxicos, sintáticos e semânticos;
- Testes com comentários em diferentes posições do código;
- Testes com variáveis declaradas e não declaradas;
- Testes com inferência correta e incorreta de tipos;
- Testes com expressões aninhadas;
- Testes com estruturas de decisão e repetição;
- Testes com todos os tipos suportados pela linguagem;
- Mensagens de erro claras e consistentes;
- Execução correta por argumento de linha de comando, sem menu interativo.

As penalidades abaixo serão aplicadas sobre os 15% desta seção:

- Menos de três arquivos de teste: **-40%**;
- Arquivos de teste com menos de dez linhas cada: **-30%**;
- Ausência de testes com erros semânticos: **-40%**;

- Ausência de testes com comentários: **-20%**;
- Ausência de testes com estruturas de controle: **-30%**;
- Ausência de testes com todos os tipos exigidos: **-30%**;
- Programa encerrando abruptamente sem mensagem clara de erro: **-30%**;
- Programa dependente de menu interativo: **-20%**.

! Important

Durante a prova de autoria, os testes poderão ser realizados com um código-fonte diferente dos arquivos entregues pelo grupo. O objetivo é verificar se o analisador semântico foi implementado de forma geral, e não apenas ajustado para os exemplos presentes no repositório.

28.10 10. Entrega

O repositório no GitHub deve conter todos os artefatos necessários para avaliar a análise semântica, a integração com as fases anteriores e a geração de código Assembly. A entrega deve deixar claro quais arquivos foram produzidos manualmente pelo grupo, quais foram gerados pelo programa e qual foi o último arquivo de teste executado.

28.10.1 10.1. Código-fonte

- Programa completo em **Python**, **C**, ou **C++**;
- Implementação das funções indicadas na Section 28.7;
- Integração com o analisador léxico da Fase 1 e com o analisador sintático da Fase 2;
- Tratamento de comentários iniciados por `*{` e terminados por `}*;`
- Construção da tabela de símbolos;
- Verificação semântica de declarações, usos de variáveis, tipos, comandos especiais, decisões e laços;
- Construção da árvore sintática atribuída;
- Geração de código Assembly para o **Cpulator-ARMv7 DEC1-SOC(v16.1)** apenas quando o programa analisado não possuir erros léxicos, sintáticos ou semânticos;
- Primeiras linhas do código contendo os integrantes do grupo, em ordem alfabética, seguidos dos usuários no GitHub e do nome do grupo no Canvas.

28.10.2 10.2. Arquivos de Teste

- Mínimo de 3 arquivos de teste com 10 linhas, ou mais, cada;
- Pelo menos um arquivo contendo apenas programas semanticamente válidos;
- Pelo menos um arquivo contendo erros semânticos intencionais;
- Pelo menos um arquivo combinando comentários, estruturas de controle, expressões aninhadas, comandos especiais e variáveis;
- Testes com literais inteiros, reais e lógicos (`bool`);
- Testes com uso correto e incorreto de variáveis;
- Testes com operações aritméticas, relacionais e lógicas;
- Testes com condições válidas e inválidas em decisões e laços;
- Testes com comentários em linhas inteiras, no final de linhas de código e entre expressões.

28.10.3 10.3. Documentação

O repositório deve conter um arquivo **README.md** bem formatado, incluindo:

- Nome da instituição de ensino, ano, disciplina e professor;
- Integrantes do grupo em ordem alfabética, seguidos dos respectivos usuários no GitHub;
- Nome do grupo no Canvas;
- Linguagem usada na implementação (**Python**, **C**, ou **C++**);
- Instruções para compilar, executar e testar o programa;
- Descrição da linguagem implementada pelo grupo;
- Descrição dos tipos suportados;
- Descrição das regras para definição e uso de variáveis;
- Exemplos de programas semanticamente válidos;
- Exemplos de programas semanticamente inválidos;
- Explicação dos arquivos de saída gerados pelo analisador.

28.10.4 10.4. Artefatos da Análise Semântica

Além do **README.md**, o repositório deve conter arquivos Markdown, texto ou JSON com os seguintes artefatos:

- Gramática atribuída em formato EBNF, usando letras minúsculas para não-terminais e maiúsculas para terminais;
- Sistema de regras de tipos em cálculo de seqüentes;
- Tabela de símbolos produzida na última execução;
- Relatório de erros semânticos da última execução, mesmo que a lista esteja vazia;
- Árvore sintática atribuída da última execução;
- Código Assembly gerado na última execução semanticamente válida;
- Indicação explícita do arquivo de teste usado para gerar os artefatos finais.

i Note

Nota: A árvore sintática atribuída deve conter, no mínimo, os tipos inferidos ou verificados para os nós relevantes, as referências à tabela de símbolos e as informações necessárias para justificar a geração do código Assembly.

28.10.5 10.5. Organização do Repositório

O repositório deve ser público e organizado de forma que seja possível identificar:

- Código-fonte do analisador;
- Arquivos de teste;
- Arquivos de documentação;
- Arquivos gerados pela última execução;
- Histórico de contribuições dos integrantes do grupo por meio de *pull requests*;
- *Commits* claros, com mensagens que descrevam as alterações realizadas.

! Important

O repositório deve ser criado exclusivamente para este projeto e deve permanecer público até o encerramento da avaliação. Alterações realizadas depois da data limite de entrega, ausência de rastreabilidade das contribuições, ou troca indevida de usuários no GitHub poderão resultar na anulação do trabalho.

28.10.6 10.6. Formato de Execução

O programa deve ser executado por linha de comando, sem menu interativo, usando o nome do arquivo de teste como argumento. Por exemplo:

```
./AnalizadorSemantico teste1.txt
```

Ou, no caso de implementação em Python:

```
python AnalizadorSemantico.py teste1.txt
```

A execução deve apresentar, no mínimo:

- Indicação do arquivo analisado;
- Resultado da análise léxica;
- Resultado da análise sintática;
- Resultado da análise semântica;
- Lista de erros encontrados, se houver;
- Caminhos dos arquivos de saída gerados.